MODULE -2

CONTENTS

I. LOAD-STORE INSTRUCTIONS

Load-store instructions transfer data between memory and processor registers.

There are three types of load-store instructions:

- Single-register transfer
- Multiple-register transfer and
- Swap

SINGLE-REGISTER TRANSFER

✓ These instructions are used for moving a single data item in and out of a register.

✓ The data types supported are signed and unsigned words (32-bit), half-words (16-bit), and bytes.

Here are the various load-store single-register transfer instructions.

```
Syntax: <LDR|STR>{<cond>}{B} Rd,addressing1
        LDR{<cond>}SB|H|SH Rd, addressing2
        STR{<cond>}H Rd, addressing2
```

Table 1: load-store single-register transfer instructions

LDR	load word into a register	*Rd <- mem32[address]*
STR	save byte or word from a register	*Rd -> mem32[address]*
LDRB	load byte into a register	*Rd <- mem8[address]*
STRB	save byte from a register	*Rd -> mem8[address]*

LDRH	load halfword into a register	*Rd <- mem16[address]*
STRH	save halfword into a register	*Rd -> mem16[address]*
LDRSB	load signed byte into a register	*Rd <- SignExtend (mem8[address])*
LDRSH	load signed halfword into a register	*Rd <- SignExtend (mem16[address])*

- ✓ LDR and STR instructions can load and store data on a boundary alignment that is the same as the data type size being loaded or stored.
  - **For example**, LDR can only load 32-bit words on a memory address that is a multiple of four bytes—0, 4, 8, and soon.

Example: This example shows a load from a memory address contained in register *r1*, followed by a store back to the same address in memory.

```
; load register r0 with the contents of the memory address pointed to by register r1
;
        LDR r0, [r1]            ; = LDR r0, [r1, #0]
;
; store the contents of register r0 to the memory address pointed to by register r1
;
        STR r0, [r1]            ; = STR r0, [r1, #0]
```

The first instruction loads a word from the address stored in register r1 and places it into register r0. The second instruction goes the other way by storing the contents of register r0 to the address contained in register r1. The offset from register r1 is zero. Register r1 is called the **base address register.**

Single-Register Load-Store Addressing Modes

The ARM instruction set provides different modes for addressing memory. These modes incorporate one of the indexing methods: pre index with write back, pre index, and post index.

Table 2: Index Methods

Index method	Data	Base address register	Example
Preindex with writeback	*mem[base + offset]*	*base + offset*	`LDR r0,[r1,#4]!`
Preindex	*mem[base + offset]*	*not updated*	`LDR r0,[r1,#4]`
Postindex	*mem[base]*	*base + offset*	`LDR r0,[r1],#4`

Note: ! indicates that the instruction writes the calculated address back to the base address register.

- ✓ *Preindex with writeback* calculates an address from a base register plus address offset and then updates that address base register with the new address.
- ✓ *Preindex* offset is the same as the preindex with writeback but does not update the address base register.
  - The preindex mode is useful for accessing an element in a data structure.
- ✓ *Postindex* only updates the address base register after the address is used.
  - The postindex and preindex with writeback modes are useful for traversing an array.

Example:

PRE r0 = 0x00000000

r1 =0x00090000

mem32[0x00009000] = 0x01010101

mem32[0x00009004] = 0x02020202

LDR r0, [r1, #4]!

Preindexing with writeback:

POST(1) r0 = 0x02020202

r1 =0x00009004

LDR r0, [r1, #4]

Preindexing:

POST(2) r0 = 0x02020202

r1 =0x00009000

LDR r0, [r1], #4

Postindexing:

POST(3) r0 = 0x01010101

r1 =0x00009004

- ✓ The above Example used a preindex method. This example shows how each indexing method affects the address held in register r1, as well as the data loaded into register r0.

The addressing modes available with a particular load or store instruction depend on the instruction class. The following Table shows the addressing modes available for load and store of a 32-bit word or an unsigned byte.

Table 3: Single-Register Load-Store Addressing, Word or Unsigned Byte

Addressing[1] mode and index method	Addressing[1] syntax
Preindex with immediate offset	`[Rn, #+/-offset_12]`
Preindex with register offset	`[Rn, +/-Rm]`
Preindex with scaled register offset	`[Rn, +/-Rm, shift #shift_imm]`
Preindex writeback with immediate offset	`[Rn, #+/-offset_12]!`
Preindex writeback with register offset	`[Rn, +/-Rm]!`
Preindex writeback with scaled register offset	`[Rn, +/-Rm, shift #shift_imm]!`
Immediate postindexed	`[Rn], #+/-offset_12`
Register postindex	`[Rn], +/-Rm`
Scaled register postindex	`[Rn], +/-Rm, shift #shift_imm`

- ✓ A signed offset or register is denoted by "+/-", identifying that it is either a positive or negative offset from the base address register *Rn*. The base address register is a pointer to a byte in memory, and the offset specifies a number of bytes.
- ✓ Immediate means the address is calculated using the base address register and a 12-bit offset encoded in the instruction.
- ✓ Register means the address is calculated using the base address register and a specific register's contents.
- ✓ Scaled means the address is calculated using the base address register and a barrel shift operation.

The following Table provides an example of the different variations of the LDR instruction.

Table 4: Examples of LDR Instructions using Different Addressing Modes

	Instruction	*r0* =	*r1* + =
Preindex with writeback	`LDR r0,[r1,#0x4]!`	`mem32[r1+0x4]`	`0x4`
	`LDR r0,[r1,r2]!`	`mem32[r1+r2]`	`r2`
	`LDR r0,[r1,r2,LSR#0x4]!`	`mem32[r1+(r2 LSR 0x4)]`	`(r2 LSR 0x4)`
Preindex	`LDR r0,[r1,#0x4]`	`mem32[r1+0x4]`	*not updated*
	`LDR r0,[r1,r2]`	`mem32[r1+r2]`	*not updated*
	`LDR r0,[r1,-r2,LSR #0x4]`	`mem32[r1-(r2 LSR 0x4)]`	*not updated*
Postindex	`LDR r0,[r1],#0x4`	`mem32[r1]`	`0x4`
	`LDR r0,[r1],r2`	`mem32[r1]`	`r2`
	`LDR r0,[r1],r2,LSR #0x4`	`mem32[r1]`	`(r2 LSR 0x4)`

The following Table shows the addressing modes available on load and store instructions using 16-bit half word or signed byte data.

Table 5: Single-Register Load-Store Addressing, Half word, signed half word, Signed Byte and Double word

Addressing[2] mode and index method	Addressing[2] syntax
Preindex immediate offset	`[Rn, #+/-offset_8]`
Preindex register offset	`[Rn, +/-Rm]`
Preindex writeback immediate offset	`[Rn, #+/-offset_8]!`
Preindex writeback register offset	`[Rn, +/-Rm]!`
Immediate postindexed	`[Rn], #+/-offset_8`
Register postindexed	`[Rn], +/-Rm`

These operations cannot use the barrel shifter. There are no STRSB or STRSH instructions since STRH store both a signed and unsigned half word; similarly STRB stores signed and unsigned bytes. The following Table shows the variations for STRH instructions.

Table 6: Variations of STRH Instructions

	Instruction	Result	*r1* +=
Preindex with writeback	`STRH r0,[r1,#0x4]!`	`mem16[r1+0x4]=r0`	`0x4`
	`STRH r0,[r1,r2]!`	`mem16[r1+r2]=r0`	`r2`
Preindex	`STRH r0,[r1,#0x4]`	`mem16[r1+0x4]=r0`	*not updated*
	`STRH r0,[r1,r2]`	`mem16[r1+r2]=r0`	*not updated*
Postindex	`STRH r0,[r1],#0x4`	`mem16[r1]=r0`	`0x4`
	`STRH r0,[r1],r2`	`mem16[r1]=r0`	`r2`

MULTIPLE-REGISTER TRANSFER

- ✓ Load-store multiple instructions can transfer multiple registers between memory and the processor in a single instruction.
- ✓ The transfer occurs from a base address register *Rn* pointing into memory.
    - Multiple-register transfer instructions are more efficient from single-register transfers for
        - moving blocks of data around memory and
        - saving and restoring context and stacks.
- ✓ Load-store multiple instructions can increase interrupts latency.
- ✓ ARM implementations do not usually interrupt instructions while they are executing.

For example, on an ARM7 a load multiple instruction takes *2 + Nt* cycles, where *N* is the number of registers to load and *t* is the number of cycles required for each sequential access to memory.

- ✓ If an interrupt has been raised, then it has no effect until the load-store multiple instruction is complete.

Compilers, such as arm cc, provide a switch to control the maximum number of registers being transferred on a load-store, which limits the maximum interrupt latency.

Syntax: <LDM|STM>{<cond>}<addressing mode> Rn{!},<registers>{^}

LDM	load multiple registers	{Rd}*N <- mem32[start address + 4*N] optional Rn updated
STM	save multiple registers	{Rd}*N -> mem32[start address + 4*N] optional Rn updated

The following Table shows the different addressing modes for the load-store multiple instructions. Here *N* is the number of registers in the list of registers.

Table 7: Addressing Mode for Load-Store Multiple Instructions

Addressing mode	Description	Start address	End address	*Rn*!
IA	increment after	Rn	$Rn + 4*N - 4$	$Rn + 4*N$
IB	increment before	$Rn + 4$	$Rn + 4*N$	$Rn + 4*N$
DA	decrement after	$Rn - 4*N + 4$	Rn	$Rn - 4*N$
DB	decrement before	$Rn - 4*N$	$Rn - 4$	$Rn - 4*N$

- ✓ Any subset of the current bank of registers can be transferred to memory or fetched from memory.
- ✓ The base register *Rn* determines the source or destination address for a load-store multiple instruction. This register can be optionally updated following the transfer. This occurs when register *Rn* is followed by the ! character, similar to the single-register load-store using preindex with write back.

Example: In this example, register *r0* is the base register *Rn* and is followed by !, indicating that the register is updated after the instruction is executed. You will notice within the load multiple instructions that the registers are not individually listed. Instead the "-" character is used to identify a range of registers. In this case the range is from register *r1* to *r3*inclusive.

Each register can also be listed, using a comma to separate each register within "{" and "}" brackets.

PRE

mem32[0x80018] = 0x03

mem32[0x80014] = 0x02

mem32[0x80010] = 0x01

r0 =0x00080010

r1 = 0x00000000

r2 = 0x00000000

r3 =0x00000000

LDMIA r0! , {r1–r3}

POST

r0 = 0x0008001c
r1=0x00000001

r2 = 0x00000002

r3 = 0x00000003

The following Figure shows a graphical representation.

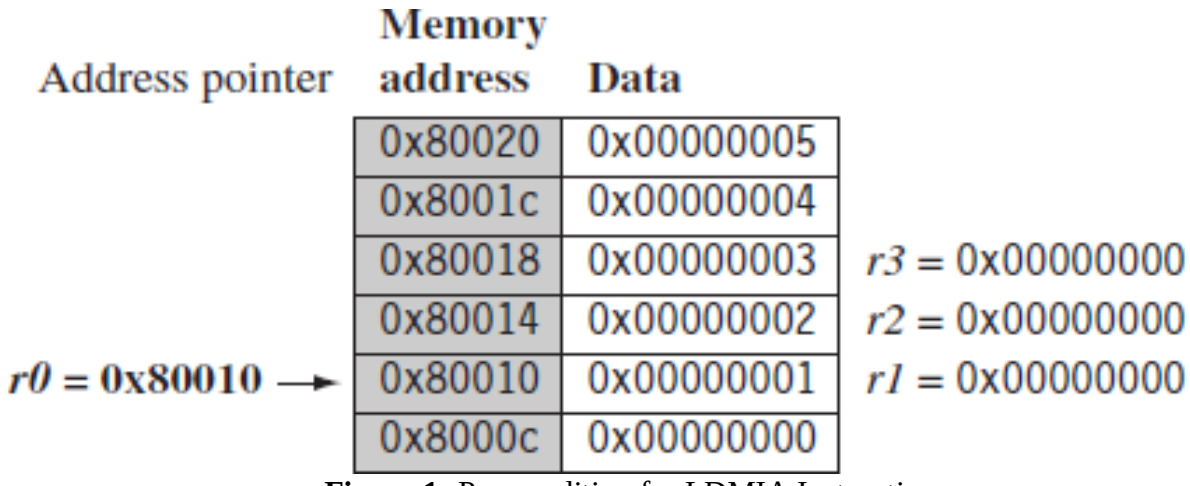

Figure 1: Pre-condition for LDMIA Instruction

- ✓ The base register *r0* points to memory address 0x80010 in the PRE condition.
- ✓ Memory addresses 0x80010, 0x80014, and 0x80018 contain the values 1, 2, and 3 respectively.
- ✓ After the load multiple instruction executes, registers *r1*, *r2*, and *r3* contain these values as shown in the following figure.

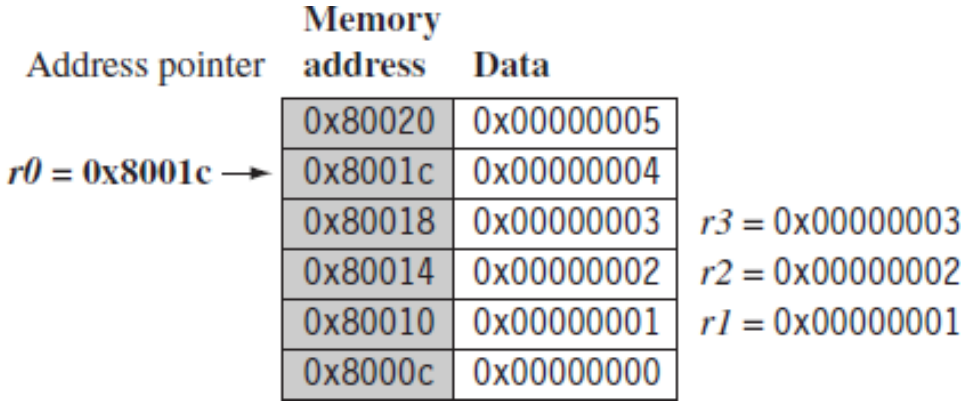

Figure 2: Post Condition for LDMIA Instruction

- ✓ The base register *r0* now points to memory address 0x8001c after the last loaded word.
- ✓ Now replace the LDMIA instruction with a load multiple and increment before LDMIB instruction and use the same PRE conditions.
- ✓ The first word pointed to by register *r0* is ignored and register *r1* is loaded from the next memory location as shown in the following Figure.

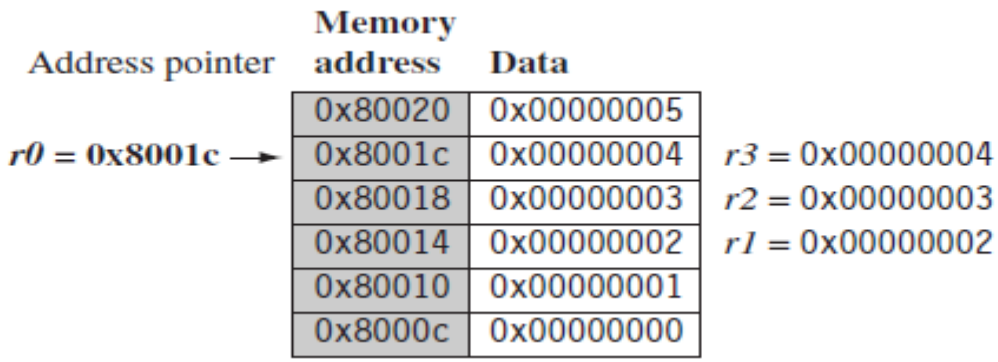

Figure 3: Post Condition for LDMIB Instruction

- ✓ After execution, register *r0* now points to the last loaded memory location. This is in contrast with the LDMIA example, which pointed to the next memory location.
    - The decrement versions DA and DB of the load-store multiple instructions decrement the start address and then store to ascending memory locations.
    - This is equivalent to descending memory but accessing the register list in reverse order.
    - With the increment and decrement load multiples; you can access arrays forwards or backwards.

- They also allow for stack push and pull operations.
- The decrement versions DA and DB of the load-store multiple instructions decrement the start address and then store to ascending memory locations.
- This is equivalent to descending memory but accessing the register list in reverse order.
- With the increment and decrement load multiples; you can access arrays forwards or backwards.
- They also allow for stack push and pull operations.

The following Table shows a list of load-store multiple instruction pairs.

Table 8: Load-Store Multiple Pairs when Base Update used

Store Multiple	Load Multiple
STMIA	LDMDB
STMIB	LDMDA
STMDA	LDMIB
STMDB	LDMIA

- If you use a store with base update, then the paired load instruction of the same number of registers will reload the data and restore the base address pointer.
- This is useful when you need to temporarily save a group of registers and restore them later.

Example: This example shows an STM *increment before* instruction followed by an LDM *decrement after* instruction.

PRE *r0 =0x00009000*

r1 = 0x00000009

r2 = 0x00000008

r3 =0x00000007
STMIB r0!, {r1–r3} MOV r1, #1
MOV r2, #2

MOV r3, #3

PRE(2) *r0 = 0x0000900c*

r1 = 0x00000001

r2 = 0x00000002

r3 =0x00000003
LDMDA r0!, {r1–r3}

POST *r0 =0x00009000*

r1 = 0x00000009

r2 = 0x00000008

r3 =0x00000007

The STMIB instruction stores the values 7, 8, 9 to memory. We then corrupt register *r1* to *r3*. The LDMDA reloads the original values and restores the base pointer *r0*.

Example: We illustrate the use of the load-store multiple instructions with a block memory copy example. This example is a simple routine that copies blocks of 32 bytes from a source address location to a destination address location.

The example has two load-store multiple instructions, which use the same increment after addressing mode.

```
        ; r9 points to start of source data
        ; r10 points to start of destination data
        ; r11 points to end of the source
loop
        ; load 32 bytes from source and update r9 pointer
LDMIA r9!, {r0–r7}
        ; store 32 bytes to destination and update r10 pointer
STMIA r10!, {r0–r7}             ; and store them
        ; have we reached the end
CMP r9, r11
BNE loop
```

- ✓ This routine relies on registers *r9*, *r10*, and *r11* being set up before the code is executed.
- ✓ Registers *r9* and *r11* determine the data to be copied, and register *r10* points to the destination in memory for the data.
- ✓ LDMIA loads the data pointed to by register *r9* into registers *r0* to *r7*. It also updates *r9* to point to the next block of data to be copied.
- ✓ STMIA copies the contents of registers *r0* to *r7* to the destination memory address pointed to by register *r10*. It also updates *r10* to point to the next destination location.
- ✓ CMP and BNE compare pointers *r9* and *r11* to check whether the end of the block copy has been reached.
- ✓ If the block copy is complete, then the routine finishes; otherwise the loop repeats with the updated values of register *r9* and*r10*.
- ✓ The BNE is the branch instruction B with a condition mnemonic NE (not equal). If the previous compare instruction sets the condition flags to not equal, the branch instruction is executed.

The following Figure shows the memory map of the block memory copy and how the routine moves through memory.

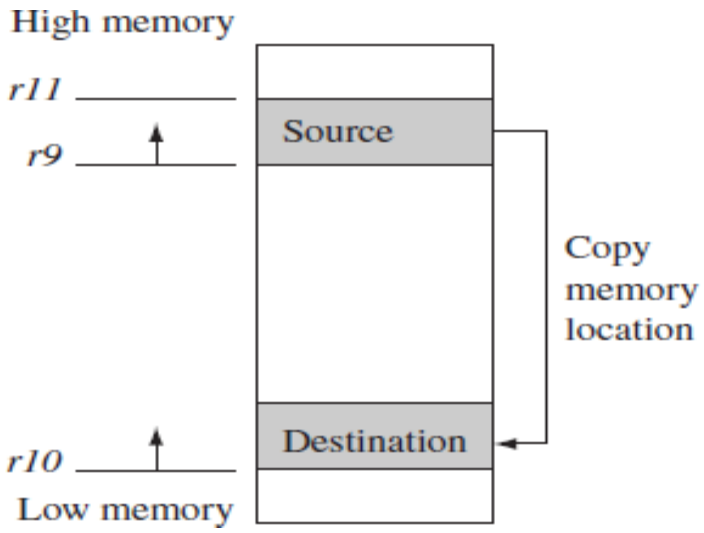

Figure 4: Block Memory Copy in the Memory map

Theoretically this loop can transfer 32 bytes (8 words) in two instructions, for a maximum possible throughput of 46 MB/second being transferred at 33 MHz. These numbers assume a perfect memory system with fast memory.

STACK OPERATION

The ARM architecture uses the load-store multiple instructions to carry out stack operations.

- The *pop operation* (removing data from a stack) uses a load multiple instruction.
- The *push operation* (placing data onto the stack) uses a store multiple instruction.

✓ When using a stack you have to decide whether the stack will grow up or down in memory.
  - A stack is either–
    - *ascending (A)* – stacks grow towards higher memory addressesor
    - *descending (D)* – stacks grow towards lower memory addresses.

✓ When you use a *full stack (F)*, the stack pointer *sp* points to an address that is the last used or full location (i.e., *sp* points to the last item on the stack).

✓ If you use an *empty stack (E)* the *sp* points to an address that is the first unused or empty location (i.e., it points after the last item on the stack).

Table 9: Addressing Methods for Stack Operations

Addressing mode	Description	Pop	= LDM	Push	= STM
FA	full ascending	LDMFA	LDMDA	STMFA	STMIB
FD	full descending	LDMFD	LDMIA	STMFD	STMDB
EA	empty ascending	LDMEA	LDMDB	STMEA	STMIA
ED	empty descending	LDMED	LDMIB	STMED	STMDA

- Next to the *pop* column is the actual load multiple instruction equivalents.
  - For example, a full ascending stack would have the notation FA appended to the load multiple instruction—LDMFA. This would be translated into an LDMDA instruction.
- ARM has specified an *ARM-Thumb Procedure Call Standard (ATPCS)* that defines how routines are called and how registers are allocated. In the ATPCS, stacks are defined as being full descending stacks. Thus, the LDMFD and STMFD instructions provide the *pop* and *push* functions, respectively.
- There are number of load-store multiple addressing mode aliases available to support stack operations (see the following Table).

Example: The STMFD instruction pushes registers onto the stack, updating the *sp*. The following Figure shows a *push* onto a full descending stack.

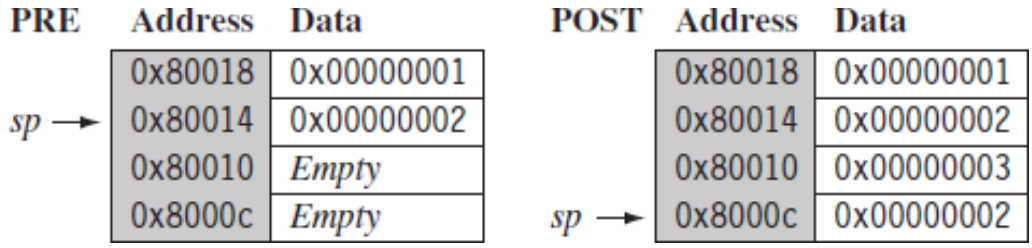

Figure 5: STMFD Instruction – Full Stack *push* Operation

You can see that when the stack grows the stack pointer points to the last full entry in the stack.

PRE *r1 =0x00000002*

r4 = 0x00000003

sp =0x00080014

STMFD sp!, {r1, r4}

POST *r1 =0x00000002*

r4 =0x00000003

sp =0x0008000c

Example: The following Figure shows a *push* operation on an empty stack using the STMED instruction.

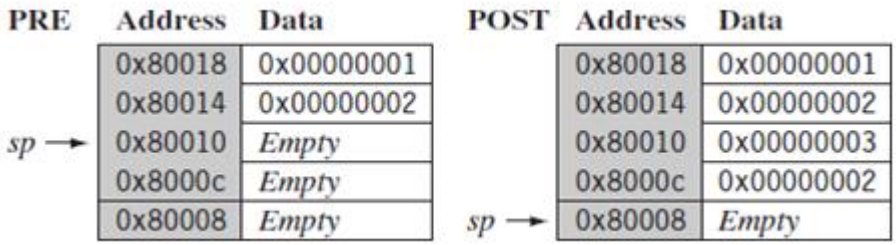

Figure 6: STMED Instruction – Empty Stack *push* Operation

The STMED instruction pushes the registers onto the stack but updates register *sp* to point to the next empty location.

PRE *r1 =0x00000002*

r4 = 0x00000003

sp =0x00080010

STMED sp!, {r1, r4}

POST *r1 =0x00000002*

r4 = 0x00000003

sp =0x00080008

- ✓ When handling a checked stack there are three attributes that need to be preserved: the stack base, the stack pointer, and the stack limit.
- ✓ The stack base is the starting address of the stack in memory.
- ✓ The stack pointer initially points to the stack base; as data is pushed onto the stack, the stack pointer descends memory and continuously points to the top of stack. If the stack pointer passes the stack limit, then a stack overflow error has occurred.
- ✓ Here is a small piece of code that checks for stack overflow errors for a descending stack:

```
; check for stack overflow
SUB sp, sp, #size
CMP sp, r10
BLLO_stack_overflow          ;condition
```

- ATPCS defines register *r10* as the stack limit or *sl*. This is optional since it is only used when stack checking is enabled.

- The BLLO instruction is a branch with link instruction plus the condition mnemonic LO.
    - If *sp* is less than register *r10* after the new items are pushed onto the stack, then *stack overflow* error has occurred.
    - If the stack pointer goes back past the stack base, then a *stack underflow* error has occurred.

SWAP INSTRUCTION

The swap instruction is a special case of a load-store instruction. It swaps the contents of memory with the contents of a register.

This instruction is an *atomic operation*—it reads and writes a location in the same bus operation, preventing any other instruction from reading or writing to that location until it completes.

```
Syntax: SWP{B}{<cond>} Rd,Rm,[Rn]
```

SWP	swap a word between memory and a register	$tmp = mem32[Rn]$ $mem32[Rn] = Rm$ $Rd = tmp$
SWPB	swap a byte between memory and a register	$tmp = mem8[Rn]$ $mem8[Rn] = Rm$ $Rd = tmp$

Swap cannot be interrupted by any other instruction or any other bus access. We say the system "holds the bus" until the transaction is complete. Also, swap instruction allows for both a word and a byte swap.

Example: The swap instruction loads a word from memory into register *r0* and overwrites the memory with register *r1*.

PRE *mem32[0x9000] = 0x12345678*

r0 =0x00000000

r1 = 0x11112222

r2 = 0x00009000

SWP r0, r1, [r2]

POST *mem32[0x9000] = 0x11112222*

r0 =0x12345678

r1 = 0x11112222

r2 =0x00009000

Example: This example shows a simple data guard that can be used to protect data from being written by another task. The SWP instruction "holds the bus" until the transaction is complete.

```
spin
        MOV r1,
        =semaphore

        MOV r2, #1
        SWP r3,r2, [r1]          ; hold the bus untilcomplete
        CMP r3, #1
        BEQ   spin
```

The address pointed to by the semaphore either contains the value *0* or *1*. When the semaphore equals *1*, then the service in question is being used by another process. The routine will continue to loop around until the service is released by the other process—in other words, when the semaphore address location contains the value *0*.

II. SOFTWARE INTERRUPT INSTRUCTION

A *software interrupt instruction (SWI)* causes a software interrupt exception, which provides a mechanism for applications to call operating system routines.

```
Syntax: SWI{<cond>} SWI_number
```

SWI	software interrupt	*lr_svc* = address of instruction following the SWI *spsr_svc* = *cpsr* *pc* = vectors + 0x8 *cpsr* mode = *SVC* *cpsr I* = 1 (mask IRQ interrupts)

When the processor executes an SWI instruction, it sets the program counter *pc* to the offset *0x8* in the vector table. The instruction also forces the processor mode to SVC, which allows an operating system routine to be called in a privileged mode.

Each SWI instruction has an associated SWI number, which is used to represent a particular function call or feature.

Example: Here we have a simple example of an SWI call with SWI number 0x123456, used by ARM toolkits as a debugging SWI. Typically the SWI instruction is executed in user mode.

```
PRE    cpsr = nzcVqift_USER
       pc = 0x00008000
       lr=0x003fffff          ;lr = r14
       r0 =0x12
0x00008000   SWI    0x123456
POST cpsr = nzcVqIft_SVC
       spsr = nzcVqift_USER
       pc = 0x00000008
       lr = 0x00008004
       r0 = 0x12
```

Since SWI instructions are used to call operating system routines, you need some form of parameter passing. This is achieved using registers. In this example, register *r0* is used to pass the parameter *0x12*. The return values are also passed back via registers. Code called the ***SWI handler*** is required to process the SWI call. The handler obtains the SWI number using the address of the executed instruction, which is calculated from the link register *lr*.

The SWI number is determined by:

*SWI_Number = <SWI instruction>**AND NOT** (0xff000000)*

Here the *SWI instruction* is the actual 32-bit SWI instruction executed by the processor.

Example: This example shows the start of an SWI handler implementation. The code fragment determines what SWI number is being called and places that number into register *r10*. You can see from this example that the load instruction first copies the complete SWI instruction into register *r10*. The BIC instruction masks off the top bits of the instruction, leaving the SWI number.
We assume the SWI has been called from ARM state.

```
SWI_handler
; Store registers r0-r12 and the link register
```

```
        STMFD sp!, {r0–r12, lr}
    ; Read the SWIinstruction
        LDR r10, [lr,#–4]
    ; Mask off top 8 bits
BIC r10, r10, #0xff000000
    ; r10 - contains the SWI number
BL service_routine
    ; return from SWI handler
LDMFD sp!, {r0–r12, pc}^
```

The number in register *r10* is then used by the SWI handler to call the appropriate SWI service routine.

III. PROGRAM STATUS REGISTER INSTRUCTIONS:

The ARM instruction set provides two instructions to directly control a *program status register (psr)*.

- ✓ The *MRS instruction* transfers the contents of either the *cpsr* or *spsr* into a register.
- ✓ The *MSR instruction* transfers the contents of a register into the *cpsr* or *spsr*.

Together these instructions are used to read and write the *cpsr* and *spsr*.

In the syntax we can see a *label* called fields. This can be any combination of *control (c)*, *extension (x)*, *status (s)*, and *flags (f)*.

```
Syntax: MRS{<cond>} Rd,<cpsr|spsr>
        MSR{<cond>} <cpsr|spsr>_<fields>,Rm
        MSR{<cond>} <cpsr|spsr>_<fields>,#immediate
```

MRS	copy program status register to a general-purpose register	*Rd = psr*
MSR	move a general-purpose register to a program status register	*psr[field] = Rm*
MSR	move an immediate value to a program status register	*psr[field] = immediate*

These fields relate to particular byte regions in a *psr*, as shown in the following Figure.

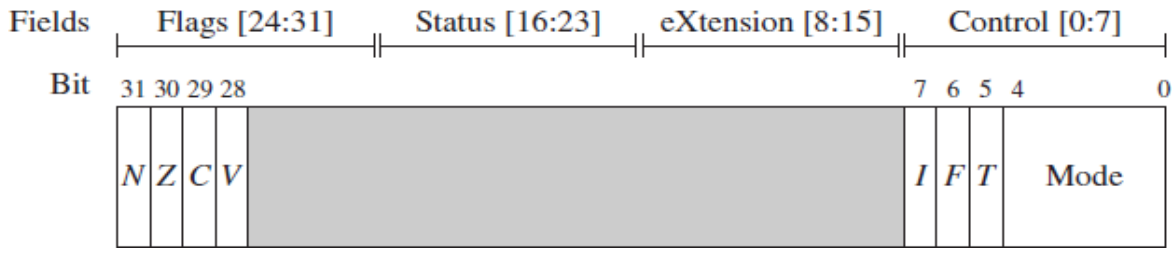

Figure 7: *psr* Byte Fields

The *c* field controls the interrupt masks, Thumb state, and processor mode.

The following example shows how to enable IRQ interrupts by clearing the *I* mask. This operation involves using both the MRS and MSR instructions to read from and then write to the *cpsr*.

Example: The MSR first copies the *cpsr* into register *r1*. The BIC instruction clears bit *7* of *r1*. Register *r1* is then copied back into the *cpsr*, which enables IRQ interrupts. You can see from this example that this code preserves all the other settings in the *cpsr* and only modifies the *I* bit in the control field.

```
PRE cpsr = nzcvqIFt_SVC
MRS   r1,cpsr
BIC r1,r1,#0x80        ;0b01000000
MSR cpsr_c, r1
POST cpsr = nzcvqiFt_SVC
```

This example is in SVC mode. In user mode you can read all *cpsr* bits, but you can only update the condition flag field *f*.